

조이한

Frontend Developer · TypeScript · React · Next.js

React · Next.js로 웹 서비스를 만드는 프론트엔드 개발자입니다. 컴포넌트 설계 시 재사용성과 확장성을 먼저 고민하고, 모노레포 환경에서 공유 UI 라이브러리를 직접 구성한 경험이 있습니다. 단순히 동작하는 것보다, 다음에도 쓸 수 있는 것을 만드는 것을 중요하게 생각합니다.



noma9363@gmail.com

010-9363-7142

서울 금천구

[turbo-portfolio-portfolio.vercel.app](#)

SKILLS

Core	TypeScript, JavaScript (ES2022+), React, Next.js 15, Recoil, Zustand
Styling	Tailwind CSS v4, Sass/SCSS, CSS Modules, MUI (Material UI)
Tooling	Turborepo, Vite, GraphQL, Dnd-kit, Atomic Design, Storybook, Framer Motion
Design	Figma, Adobe Illustrator, Photoshop, InDesign
Etc	Git, GitHub, Vercel, IntelliJ IDEA

PROJECTS

2023.08 — 2023.11

딤홈 쇼핑몰

SOSTeam

개인 쇼핑몰 플랫폼 · 서버 6명 · 웹 3명 · 디자이너 1명 · 고객사 협업

프론트엔드 · 테마

React TypeScript MUI GraphQL Recoil Dnd-kit Atomic Design Sass

- Atomic Design 패턴 기반 컴포넌트 아키텍처 설계 및 구축
- 상품 등록 페이지 전담 구현 (상품 정보 · 이미지 · 카테고리 영역)
- Dnd-kit으로 상품 이미지 드래그앤드롭 순서 변경 구현, Recoil 전역 상태와 실시간 동기화
- GraphQL 쿼리 연동으로 공지사항 필터 검색 기능 구현
- 관리자 페이지 레이아웃 (문의 · 관리자 페이지 컴포넌트) 담당
- 프로젝트 전체 UI 테마 설계 및 글로벌 스타일 관리

TROUBLE SHOOTING

- TypeScript 인터페이스 설계** — 팀원들의 Product 관련 인터페이스에 공통 속성이 반복되는 구조를 파악, 공통 속성을 담은 ProductLayer Interface를 선제적으로 설계 후 각 Props가 extends하도록 구조화 → 중복 제거 및 확장성 확보
- MUI DatePicker classes 충돌** — 외부 classes props가 내부 기본 스타일을 덮어쓰는 문제. 스프레드 방식으로 명시적 병합(...props.classes)하여 해결
- 레이아웃 콘텐츠 영역 패딩 제어** — Stack 안에 {children}을 직접 두면 레이아웃에서 콘텐츠 패딩을 제어할 수 없는 구조. children을 Box로 감싸고 styles.content 클래스를 적용해 부모 레이아웃에서 간격을 일관되게 관리

2024.06 — 진행 중

Lane 게임 공략 포럼

SOSTeam

SVG · CSS 좌표 기반 게임 공략 사이트 · 서버 6명 · 웹 4명 · 디자이너 1명

프론트엔드 · 테마

React TypeScript SVG CSS

- SVG 좌표 시스템을 활용한 게임 맵 공략 인터랙티브 UI 구현
- 사이트 초기 레이아웃 구축 및 전체 프로젝트 테마 담당
- 11인 규모 팀에서 프론트엔드 리드 역할 수행

2025.10 — 진행 중

포트폴리오 사이트 + Taski

개인 프로젝트

Turborepo 기반 모노레포 · 포트폴리오 · 칸반 일정 관리 앱

1인 개발

Next.js 15 TypeScript Tailwind CSS v4 Framer Motion Turborepo Shadcn/ui Zustand Dnd-kit

Storybook

- Turborepo + pnpm workspaces 모노레포 설계, @repo/ui 공유 컴포넌트 패키지 분리 — 여러 앱이 동일 라이브러리를 공유하는 확장 가능한 구조 구현

- Tailwind CSS v4 마이그레이션 — @theme / @utility 블록 기반 토큰 시스템, 앱별 독립 테마 관리
- Framer Motion 스크롤 애니메이션 및 stagger 효과 적용, Storybook 8 컴포넌트 개발 환경 구성
- **Taski** — 칸반 보드 일정 관리 앱. 카테고리 관리, 드래그앤드롭 우선순위 정렬, Zustand 전역 상태 관리 구현
- Vercel 자동 배포 파이프라인 구축 (main ← develop PR 머지)

TROUBLE SHOOTING

- **Tailwind v4 모노레포 클래스 purge** — packages/ui 클래스가 빌드 시 제거되는 문제. globals.css에 @source 지시자로 UI 패키지 경로를 추가하여 해결
- **모노레포 TypeScript 컴파일 오류** — Next.js가 @repo/ui TS 소스를 해석 못하는 문제. transpilePackages: ['@repo/ui'] 설정으로 해결

EDUCATION

2021.06 — 2023.06	한양사이버대학교 뉴미디어디자인학과 졸업 (편입) · 복수전공: 응용소프트웨어 · 학점 3.84 / 4.5
2014.03 — 2017.03	한강미디어고등학교 산업디자인과 졸업

CERTIFICATE

2020.11	GTQi (그래픽기술자격 일러스트) 1급 · 한국생산성본부
---------	----------------------------------

LINKS

GitHub	github.com/Noma9363/turbo-portfolio
Portfolio	turbo-portfolio-portfolio.vercel.app 개인 포트폴리오 사이트
Taski	turbo-portfolio-taski.vercel.app 칸반 일정 관리 앱 · Zustand · Dnd-kit
Figma	Sennheiser 사이트 리뉴얼 팀프로젝트 발표자료 팀 3인

자 기 소 개 서

프론트엔드 개발자 지원 · 조이한 · noma9363@gmail.com

1 디자인과 개발 사이에서, 프론트엔드를 선택한 이유

저는 산업디자인을 전공한 뒤 뉴미디어디자인과 응용소프트웨어를 복수전공하며 자연스럽게 디자인과 개발 사이의 접점을 탐구해 왔습니다. Figma로 시안을 그리고 직접 코드로 구현하는 과정에서, 내가 만든 것이 화면에 살아 움직이는 순간의 즐거움을 알게 되었습니다. 그 즐거움이 지금도 저를 프론트엔드 개발로 이끄는 가장 강한 동기입니다.

단순히 주어진 디자인을 마크업하는 것을 넘어, **사용자 경험을 코드로 설계하는 개발자**가 되고 싶다는 목표로 TypeScript, React, Next.js를 깊이 있게 학습해 왔습니다. 포트폴리오 사이트를 Turborepo 기반 모노레포로 직접 설계한 것도, 실무에서 통하는 아키텍처 감각을 직접 체득하고 싶어서였습니다.

2 팀 프로젝트에서 배운 것 — 코드 품질과 협업

SOSTeam 동아리에서 **딴짱 쇼핑몰** 프로젝트를 진행하며 처음으로 11인 규모의 협업을 경험했습니다. React + TypeScript + GraphQL 스택을 사용했고, 저는 프론트엔드와 프로젝트 테마를 담당했습니다.

가장 기억에 남는 것은 **TypeScript 인터페이스를 선제적으로 설계**했을 때입니다. 팀원들이 작성한 Product 관련 인터페이스들을 보다 보니 공통 속성이 반복되는 구조가 눈에 띄었습니다. 이대로 두면 나중에 한 속성을 바꿀 때 여러 타입을 모두 찾아 수정해야 하는 비용이 생긴다고 판단했고, 공통 속성을 담은 `ProductLayerInterface`를 먼저 설계한 뒤 `ProductCategoryProps`, `ProductContentProps`, `ProductInfoProps`가 이를 extends하도록 구조를 잡았습니다. 이 경험을 통해 "지금 동작한다"와 "나중에도 안전하다"는 다르다는 것을 배웠습니다.

3 스스로 설계하고 배포까지 — 모노레포 개인 프로젝트

팀 프로젝트 외에 개인적으로 Next.js 15 + Tailwind CSS v4 기반 포트폴리오 사이트와 칸반 일정 관리 앱 **Taski**를 Turborepo 모노레포로 구축하여 Vercel에 배포했습니다. 단순한 포트폴리오를 넘어, **실무 아키텍처를 직접 구현해보는 실험**이었습니다.

이 과정에서 예상치 못한 문제를 만났습니다. `apps/portfolio`에서 `packages/ui`의 Tailwind 클래스가 빌드 시 purge되어 스타일이 사라지는 것이었습니다. 원인은 Tailwind가 기본적으로 현재 앱의 소스만 스캔하기 때문이었고, `globals.css`에 `@source` 지시자로 UI 패키지 경로를 명시하여 해결했습니다. 에러 메시지 하나 없이 스타일만 증발하는 문제라 원인 파악에 시간이 걸렸지만, 덕분에 Tailwind v4의 빌드 파이프라인을 깊이 이해하게 되었습니다.

Taski에서는 딴짱 쇼핑몰에서 배운 드래그앤드롭 패턴을 다시 적용하되, 이번에는 상태 관리 라이브러리를 Recoil 대신 **Zustand**로 선택했습니다. 보일러플레이트가 적고 구조가 단순한 Zustand가 개인 프로젝트 규모에 더 적합하다고 판단했고, 두 라이브러리의 트레이드오프를 직접 비교하며 상황에 맞는 기술 선택의 감각을 키웠습니다.

4 배움을 멈추지 않는 개발자

저는 아직 신입이고, 모르는 것이 많다는 것을 압니다. 그래서 더욱 솔직하게, 그리고 적극적으로 배우려 합니다. 팀 프로젝트에서 코드 리뷰 의견을 받으면 왜 그런지 이해하고 나서 수정하는 습관을 들여왔습니다. 단축 네이밍이 가독성을 해친다는 피드백을 받은 이후 PR 커밋 메시지부터 변수명까지 명시적으로 쓰는 것을 원칙으로 삼게 된 것도 그 과정에서 생긴 변화입니다.

디자인을 알고, 사용자 시각에서 UI를 바라보며, 구체적인 트러블슈팅 경험이 있는 신입 프론트엔드 개발자로서 — **빠르게 팀에 기여하고, 함께 성장하고 싶습니다.** 기회를 주신다면 최선을 다하겠습니다.

